Accuracy Constrained Compilation

Built, certified and measured: one sound error budget spent across four kinds of approximation

THE RESULT

6.1×

Image kernel at β = 0.01, against 1.8× for the precision-only tuner.

0 of 46,080 sampled runs exceeded their certified bound.

Anand B
24BCE2199

Sathguruvasan Arunothayan
24BCE2231

Karthik Senthilkumar
24BCE2205

02

Agenda

03 – 05	**The Promise, Kept**	What Review 1 asked for and how each objective closed
06 – 11	**The System as Built**	Language, SSA IR, error engine, four levers, allocator, code generation
12 – 16	**The Results**	Six benchmarks, the soundness figure, Pareto curves, native timings
17 – 18	**Risks and Limits**	Which predicted risks materialised, and what we learned
19 – 21	**Demo and Close**	The working frontend, how to run it, conclusion

03

In One Sentence

Declare how wrong the answer may be.
The compiler returns the fastest program
it can prove stays inside that limit.

46,080

sampled executions checked against their certified bound

0

violations: no measured error ever exceeded its proof

6 / 6

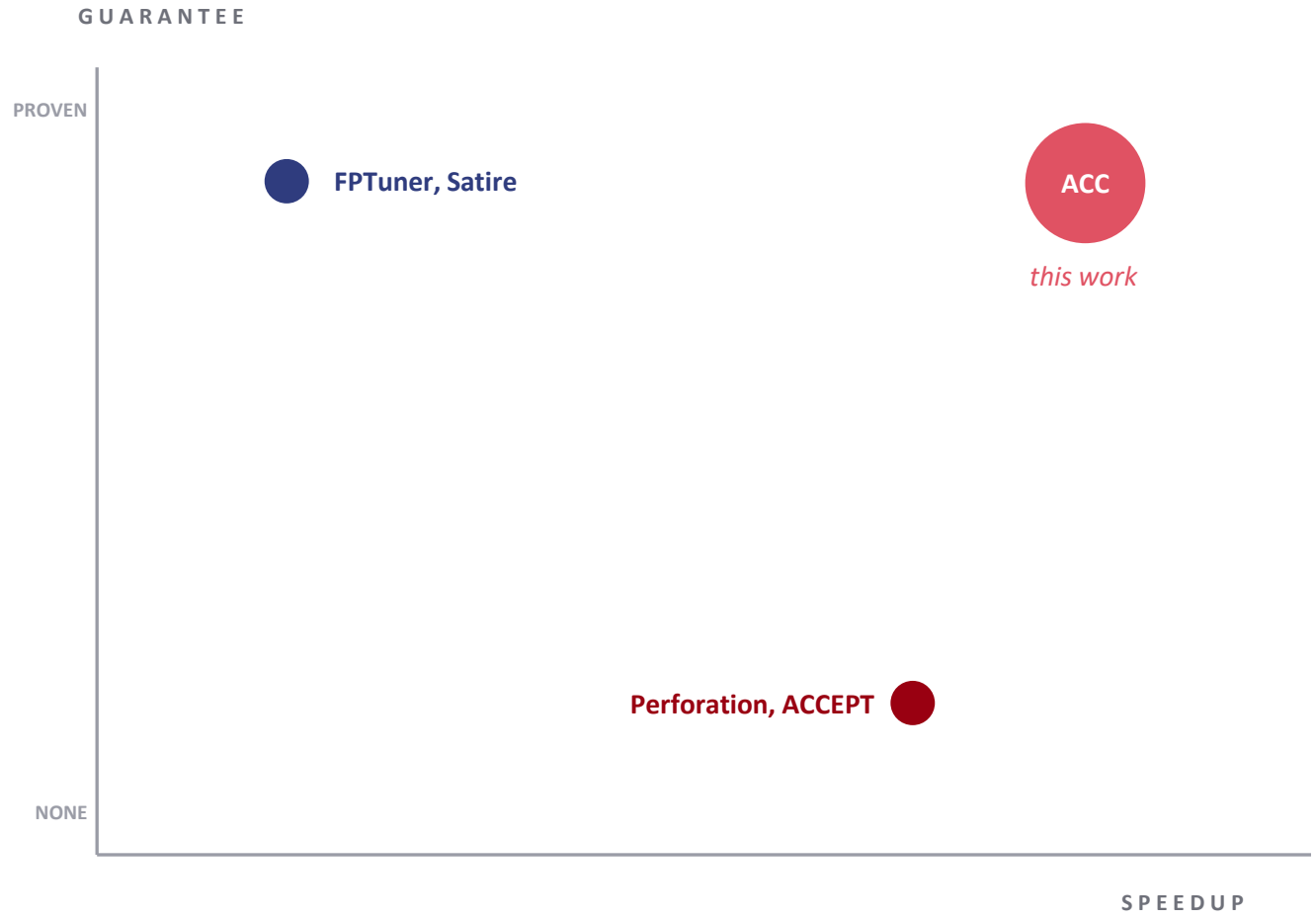
kernels whose emitted C reproduces the interpreter bit for bit

At Review 1 this was a promise. It is now a working compiler with a frontend.

04

The Empty Corner, Filled

Review 1 placed every prior system on two axes. Here is where the built compiler lands.



PROVEN

Every returned configuration carries a certified bound; 46,080 samples, 0 violations.

BROAD

Four levers in one optimisation: precision, perforation, memoization, tables.

FASTER

6.1× image, 3.3× decay, 3.1× k-means (modeled); up to 9.8× measured natively.

BEATS THE BASELINE

Precision-only tuner on the same budgets: 1.8×, 1.8×, 1.0×.

05

Objectives Scorecard

The six objectives from Review 1, each with the evidence that closes it.

01 Define the language

DONE

Numeric kernels: scalars, n-d arrays, counted loops, pure functions, a budget annotation. Lowered to SSA with phis.

02 Build the error engine

DONE

Affine ranges plus affine error forms. Four error kinds become one currency and compose by addition.

03 Implement the menu

DONE

Precision (6 formats), perforation (3 modes), memoization (3 cell sizes), tables (5 sizes).

04 Model every cost

DONE

Per-op costs calibrated on this host with a gcc microbenchmark; tables, caches and conversions priced.

05 Solve the allocation

DONE

Exact ILP (HiGHS) on the multiple choice knapsack in ≈3 ms; simulated annealing fallback.

06 Prove it empirically

DONE

46,080 sampled runs, 0 violations; 71 automated tests, all passing.

Objective five was the research. It is solved exactly, in milliseconds, at the sizes these kernels produce.

06

Architecture: Every Phase Built

Front to back, 3,500 lines of Python plus a C back end. The optimizer carries the contribution.

Lexical analysis — `lexer.py`

›

Syntax analysis — `parser.py`

›

Semantic analysis — `semantic.py`

›

SSA IR + reductions — `ir.py`

›

Unroll to dataflow graph — `ddg.py`

↓

Affine range analysis — `analysis.py`

›

Error models, 4 levers — `levers.py`

›

Allocate ⇄ certify — `allocator.py · certify.py`

›

Code generation (C) — `codegen.py`

›

Verify: Monte Carlo + gcc — `interp.py · cbench.py`

AROUND THE COMPILER

Flask JSON API · browser frontend (editor, budget slider, lever toggles, six analysis views) · 71 pytest cases · host cost calibration

Navy boxes are the research. White boxes are standard compiler construction, all of it built rather than borrowed.

07

The Language and Its SSA Form

A source kernel and the IR the compiler reasons about. Reduction loops are detected here.

SOURCE · KMEANS.ACC

```
kernel kmeans {
  budget 1e-2;
  input p[16, 8] in [0, 1];
  input c[4, 8] in [0, 1];
  output w[16, 4];
  fn kern(d) = exp(-d) * (1 + d);

  for i in 0..16 {
    for j in 0..4 {
      let d = 0;
      for k in 0..8 {
        let t = p[i, k] - c[j, k];
        d += t * t;
      }
      w[i, j] = kern(d);
    }
  }
}
```

SSA IR (INNER LOOP)

```
L3: for k in 0..8 {    ; reduction (perforable)
  d.1 = phi(d.0, d.2)
  t1  = load p[i, k]
  t2  = load c[j, k]
  t.0 = sub t1, t2       ; site s4
  t3  = mul t.0, t.0     ; site s5
  d.2 = add d.1, t3      ; site s6
}
t4 = call kern(d.1)      ; site s7
store w[i, j] = t4
```

ONE DEFINITION, ONE ERROR TERM

Every site is a static instruction; its dynamic instances share one choice.

COUNTED LOOPS UNROLL EXACTLY

k-means becomes 1,761 dataflow nodes; ranges and errors are tracked per instance.

Semantic checks reject what cannot be bounded: out-of-range indices, possible division by zero, log of a possibly non-positive value.

08

The Error Engine

Two affine analyses: one for the value of every node, one for its error.

$$E = \Sigma\, c_k \eta_k + r, \quad \eta_k \in [-1, 1], \quad r \geq 0 \qquad \textbf{bound} = \max_j \left(\Sigma |c_k| + r\right) \leq \beta$$

LINEAR OPERATIONS

Coefficients stay exact. Errors with opposite signs cancel, which intervals can never do (x − x is exactly 0).

NONLINEAR OPERATIONS

Mean value forms evaluated over the approximate operand range, so the result is a bound, not a first order estimate.

SIGN-UNCERTAIN TERMS

When an operand's range straddles zero the coefficient's sign is unknown; that part moves to r and can no longer cancel.

EACH LEVER'S LOCAL ERROR ENTERS AS A FRESH SYMBOL

Rounding

$u \cdot |\hat{x}| + d$

Perforation

$\max|\text{skipped term}|$

Memo

$L \cdot \delta + \text{own}$

Table

$h^2/8 \cdot \max|f''|$

This is the common currency from Review 1, implemented: four kinds of error, one object, composed by addition.

09

The Four Levers, As Built

Each option carries a sound error model, a cost, and a bit-faithful emulation.

Lever	Options offered	Error model	Emulated / emitted as
Precision	fp64 · fp32 · fp16 · bf16 · fp8 E4M3 · block FP	$u \cdot \lvert\hat{x}\rvert$ + subnormal floor; block FP from a static shared exponent	Round-to-nearest-even on operands and result
Perforation	skip odd iterations · drop last 25% · drop last 50%	$\Sigma \max\lvert\text{skipped term}\rvert$ per instance, from proven ranges	New loop bounds or stride in C
Memoization	cache cells of width range/256 … /4096	$L \cdot \delta$ with $L = \max\lvert f'\rvert$ over the cell, plus fp32 call error	Per-site cache, first input in a cell fills it
Tables	64 · 256 · 1024 · 4096 · 16384 entries	$h^2/8 \cdot \max\lvert f''\rvert$ + fp32 rounding of entries and interpolation	fp32 table literal + linear interpolation

Options that would overflow a format, leave a table's domain or exceed a block exponent are rejected by the certifier, never shipped.

10

Allocate, Then Certify

The knapsack is solved on a linear model; every answer is re-proved before it is accepted.

$$
\begin{aligned}
\text{minimise} \quad & \Sigma\ \text{cost}(c,t)\ \cdot\ x(c,t) \\
\text{subject to} \quad & \Sigma\ \text{err}(c,t)\ \cdot\ x(c,t)\ \leq\ \beta' \\
& \Sigma_t\ x(c,t)\ =\ 1 \quad \text{per site } c \\
& x(c,t)\ \in\ \{0,\ 1\}
\end{aligned}
$$

1. **Solve**
   Exact ILP via HiGHS. Simulated annealing with greedy seeding beyond 4,000 options.
2. **Certify**
   Propagate error forms for the chosen configuration: the true, sound bound.
3. **Adjust β′**
   Bisect the budget handed to the solver until the certified bound is as close to β as possible.

MEASURED ON THE SIX BENCHMARKS

14–40
candidate options per kernel

2.1–2.8 ms
per ILP solve

0.81 s
slowest full compile, incl. 1,024-sample verification

The Review 1 subtlety, resolved: the linear model may be wrong, but it can only cost speed, never soundness, because nothing ships uncertified.

Code Generation and Cross-Check

One C source, two builds. The certified program is the program that is emitted.

`-DACC_EMULATE`

Every rounding emulated in double precision: fp16, bf16, fp8, block FP, tables, caches.

Compared with the Python interpreter on the same inputs: max difference 0.0 on all 6 kernels.

`Native build`

fp32 arithmetic, real loop bounds, real tables and caches, gcc -O2, best of 21 interleaved runs.

Times the structural levers on this machine. Precision-only changes compile to identical code on x86.

Kernel	Structural levers	Baseline	Approximated	Native speedup
FIR filter	none	98 ns	97 ns	**1.01×**
k-means distance	table	2930 ns	299 ns	**9.79×**
Dense layer	table	201 ns	196 ns	**1.03×**
Newton iteration	none	47 ns	47 ns	**1.00×**
Image kernel	table	4611 ns	615 ns	**7.49×**
Decaying impulse	perforation	218 ns	119 ns	**1.83×**

12

Results Across Six Benchmarks

Each kernel at its declared budget. Speedups are against unmodified fp32; the precision-only tuner is our own allocator restricted to lever one.

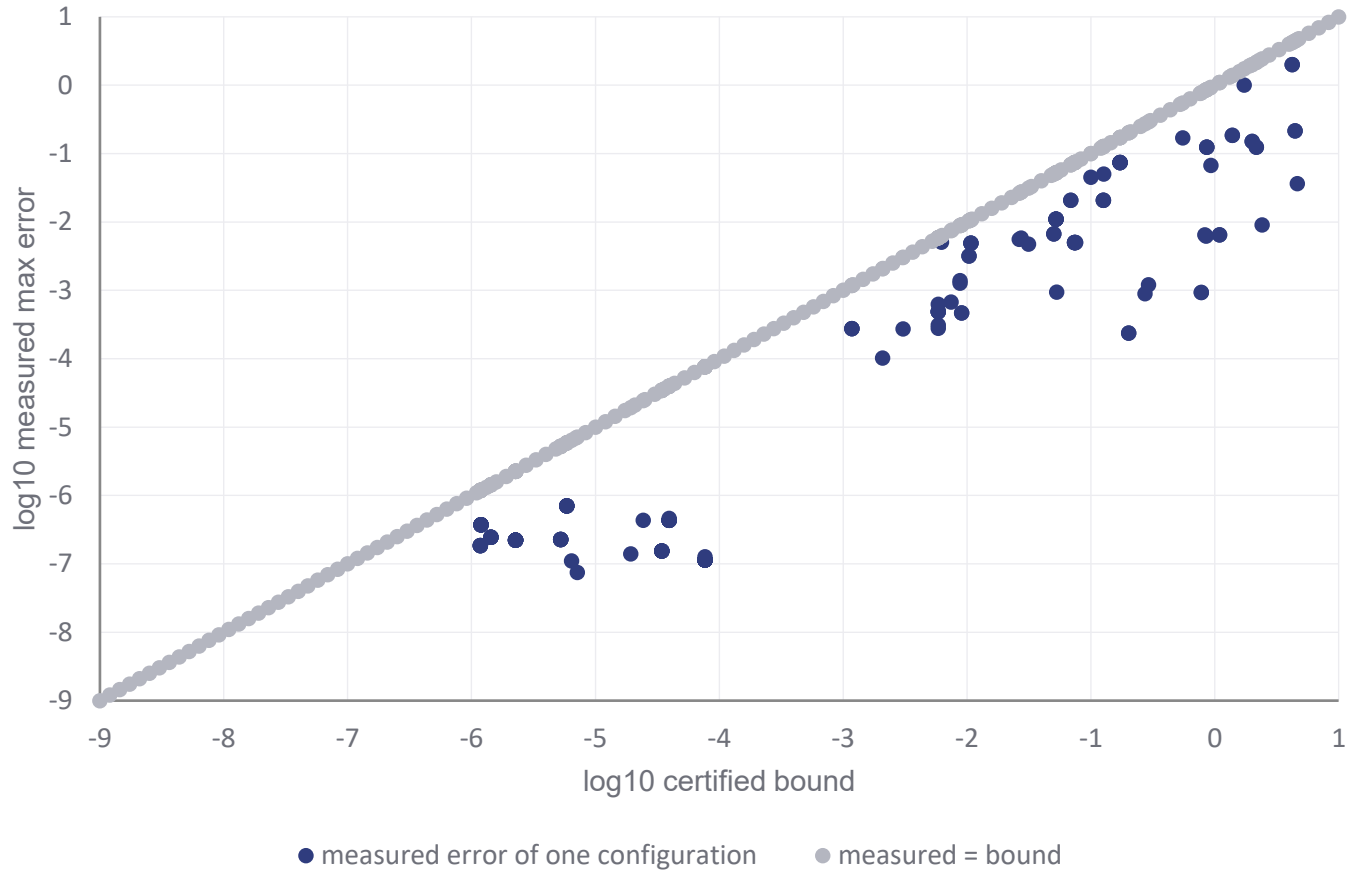
Kernel	Budget β	Certified	Measured	Bound ÷ measured	Speedup	Precision only	Levers chosen
FIR filter	0.02	5.9e-3	6.3e-4	9.4×	**1.21×**	1.21×	precision
k-means distance	0.01	3.9e-5	4.6e-7	85.0×	**3.09×**	1.00×	table
Dense layer	0.01	5.9e-3	3.1e-4	18.9×	**1.65×**	1.37×	precision + table
Newton iteration	1e-3	7.7e-5	1.3e-7	603.8×	**1.00×**	1.00×	none (fp32)
Image kernel	0.01	8.8e-3	1.4e-3	6.3×	**6.09×**	1.78×	precision + table
Decaying impulse	0.02	0.011	4.9e-3	2.2×	**3.27×**	1.83×	perforation + precision

Every row is sound. Where a structural lever applies, the joint allocator beats precision alone by 3.1× (k-means), 3.4× (image) and 1.8× (decay).

13

The Figure We Had to Produce

Review 1: “every sampled point must fall below the line.” Each dot is one certified configuration from the budget sweeps.



WHAT IT SHOWS

154

certified configurations plotted

0

points above the diagonal

2.2× – 604×

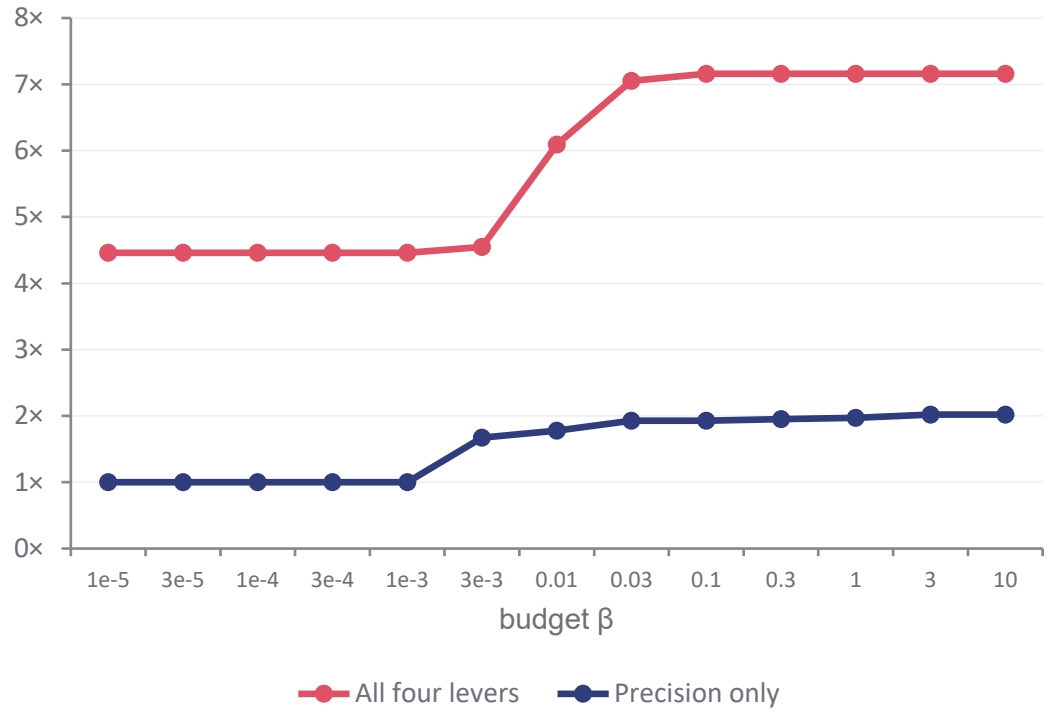
bound ÷ worst measured error at the headline budgets

No violation. The distance below the line is the price of soundness: tight for linear kernels, loose for long nonlinear chains.

Pareto Curves: Joint Allocation Wins

Modeled speedup against the budget. Both curves are certified at every point and monotone by construction.

IMAGE KERNEL · IMAGE.ACC



DECAYING IMPULSE · DECAY.ACC

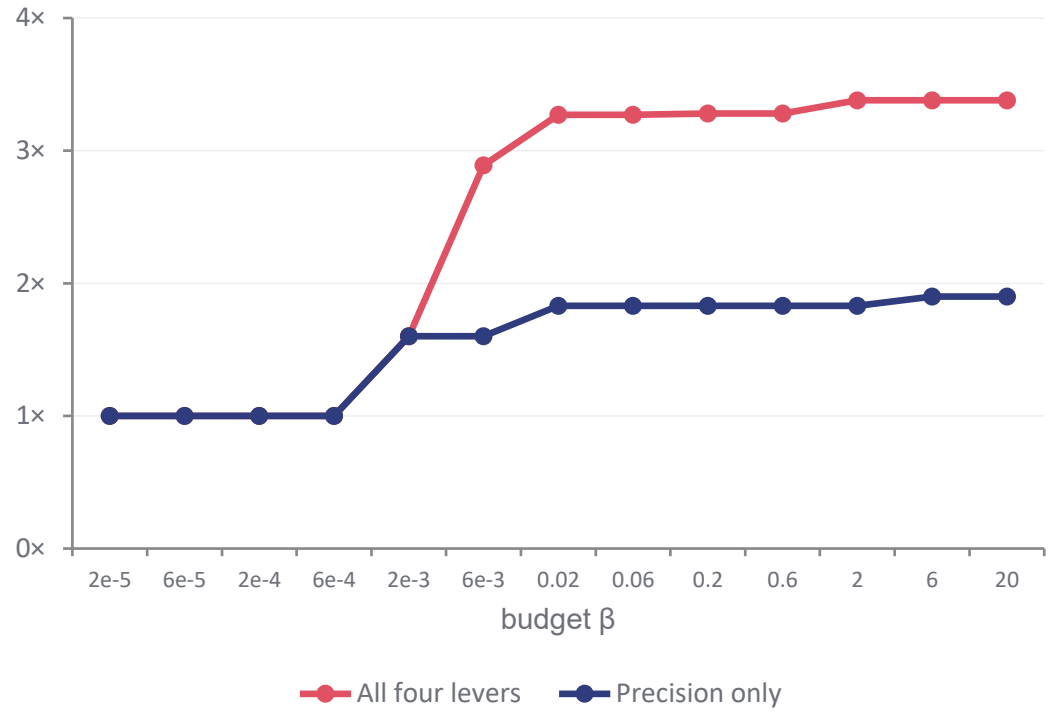
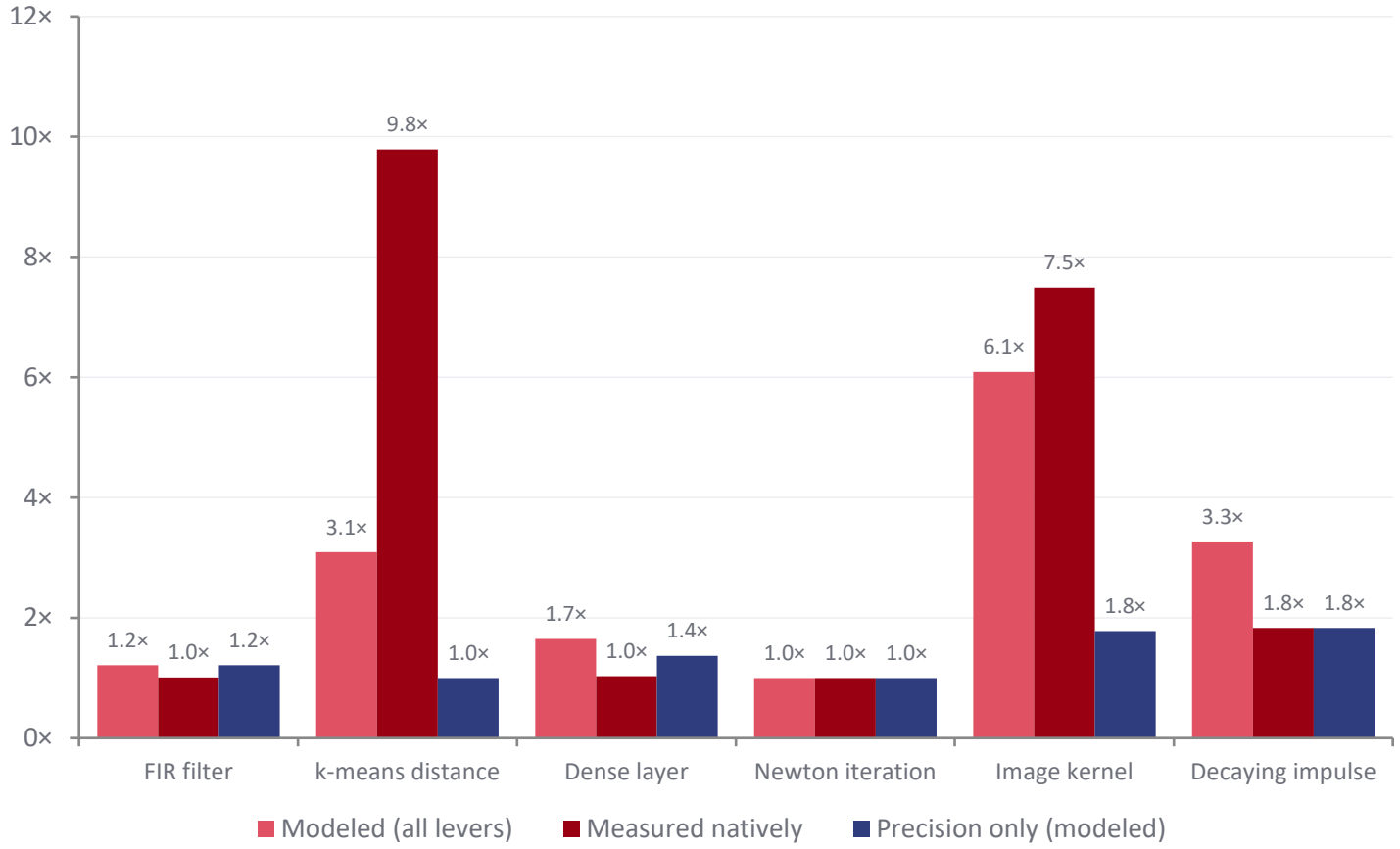


Image tops out at 7.2× with a table plus fp16 (precision alone: 2.0×). Decay reaches 3.4× once perforation unlocks (precision alone: 1.9×).

15

Modeled Versus Measured Speedup

Modeled speedup comes from the calibrated cost model; measured speedup is the native gcc build on this laptop.



TABLES BEAT THE MODEL

libm's expf and tanhf are slower on this host than the latency model assumes, so k-means measures 9.8× and image 7.5×.

PRECISION DOES NOT SHOW NATIVELY

x86 scalar units have no faster fp16 or fp8 path; that gain needs vector or matrix units and is modeled only.

PERFORATION LANDS AS PREDICTED

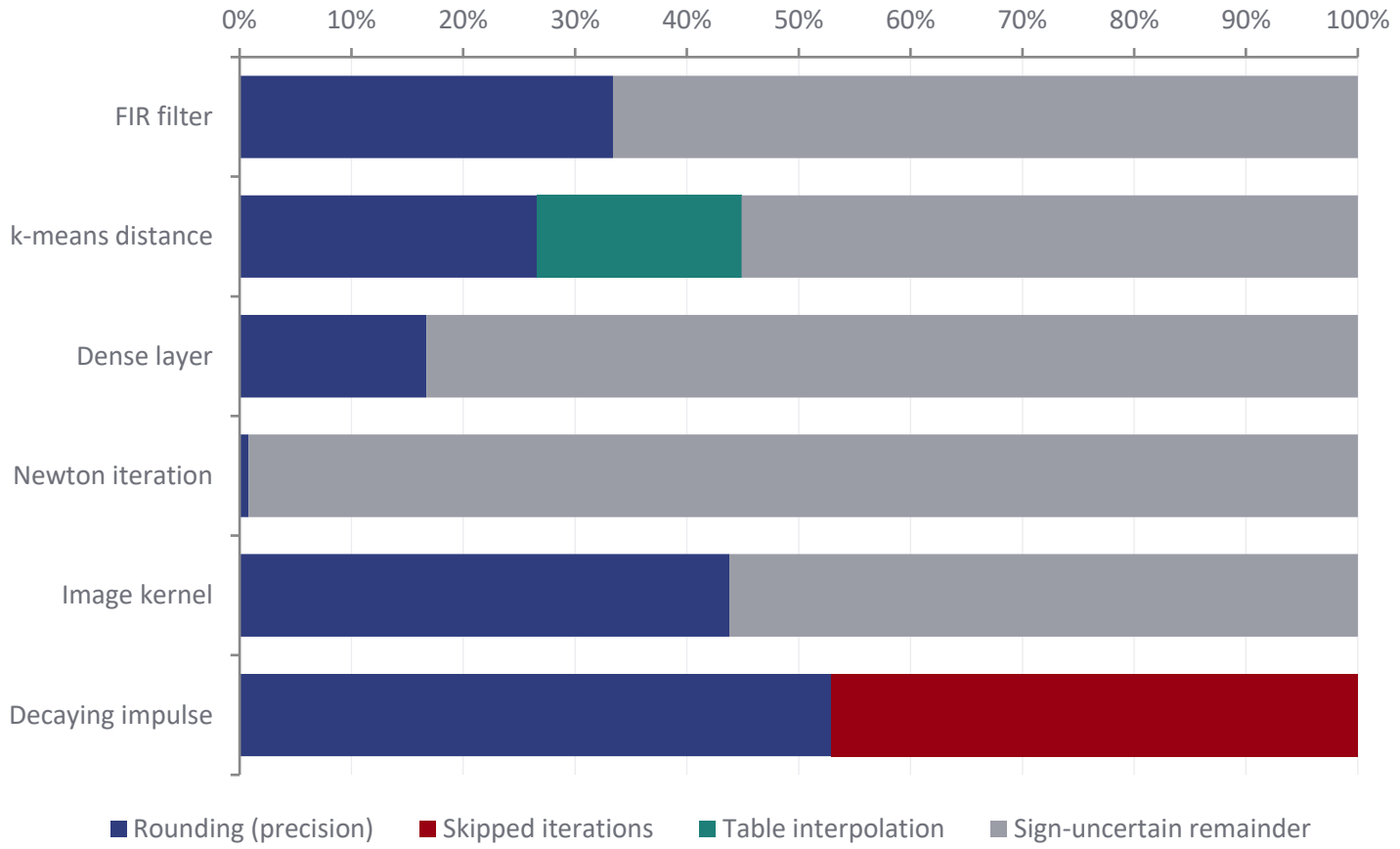
Decay: 3.3× modeled, 1.8× measured, the gap being the precision share.

Structural levers deliver real speed today. Precision gains are honest model numbers that need vector hardware to be realised.

16

Where Each Budget Went

Composition of the certified bound on the worst output, by error kind.



THE GREY BAR IS THE FRONTIER

Sign-uncertain terms dominate FIR, dense and Newton: centred input ranges make product signs unknown, so those errors cannot cancel.

DECAY SPENDS IT EVENLY

Half the budget buys fp16, half buys dropping the tail. The allocator found the split on its own.

TABLES ARE NEARLY FREE

On image the 4,096-entry table costs 3.8e-7 of a 0.01 budget.

The breakdown is shown live in the frontend for every compile, per lever and per site.

Risks Revisited

The three risks named at Review 1, and what actually happened.

PREDICTED HIGH

Bounds loosen through long chains

MATERIALISED

Newton: fp32 measures 1.3e-7, but we can only prove 7.7e-5 (604× loose). At β = 1e-6 the compiler must pay for fp64 (0.51×). The negative result Review 1 predicted, with its cause.

PREDICTED MEDIUM

Block format error is data dependent

MITIGATED

Shared exponent fixed statically from the proven range entering the block, as proposed. Forced onto every site of five kernels: sound in all tests.

PREDICTED MEDIUM

ILP does not scale

NOT REACHED

At most 40 options per kernel, solved in ≈3 ms. Annealing fallback implemented and selectable in the UI, never needed.

The first risk is real and now quantified. It is the clearest direction for future work: relational or probabilistic bounds for iterative kernels.

What We Learned, What Is Left

Findings we did not predict, and the boundaries of the current system.

FINDINGS

Tables dominate memoization

Under our cost model a table is cheaper than a cache whenever both meet the error bound. Memoization is implemented, certified and tested, but never selected.

Calibration matters

This host's libm is several times slower than a textbook model; calibrating per machine changed which lever wins.

The remainder is the real bottleneck

Most looseness comes from sign-uncertain terms, not from the levers' own error models.

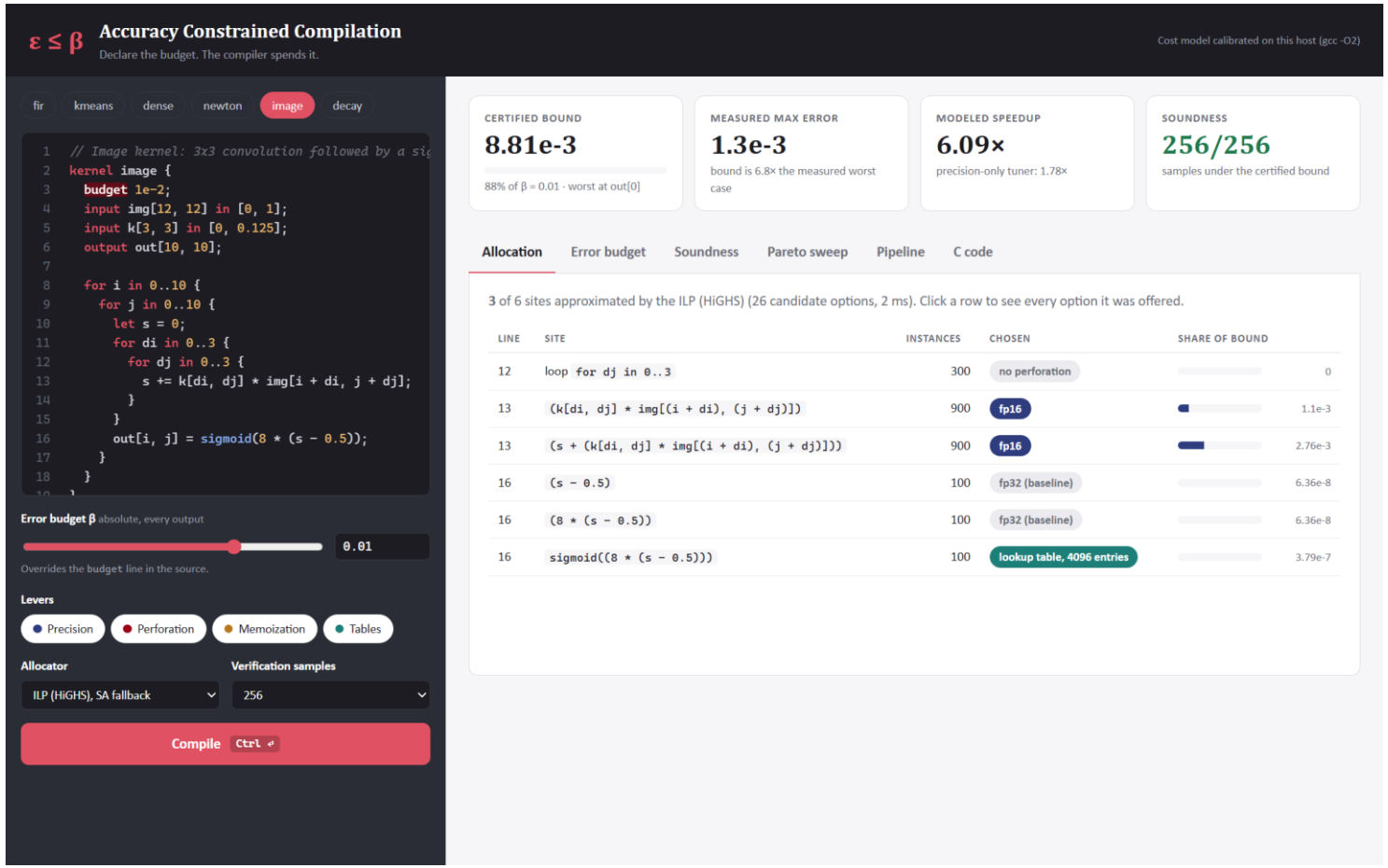
LIMITS

- No pointers, recursion, data-dependent loops or concurrency: the Review 1 scope, kept deliberately.
- Programs are unrolled for analysis; we cap at 60,000 operations.
- Precision speedups are modeled; native gains need SIMD or tensor units.
- The bound is against real arithmetic; verification compares with fp64, a 1e-16-relative proxy.
- Block FP uses one static exponent per operand rather than per hardware block.

Next steps: per-iteration contraction analysis for iterative kernels, vectorised code generation to realise precision gains, and real block FP.

Demo: The Compiler in a Browser

Edit a kernel, drag the budget, toggle levers. Each compile runs every phase, certifies, and verifies.



Allocation

every site, its chosen option, and its share of the bound

Error budget

the bound split by lever, plus the top contributing sites

Soundness

per-sample measured error against the certified line

Pareto sweep

13 budgets, full system against precision only

Pipeline

phase timings, AST, SSA IR, allocator trace

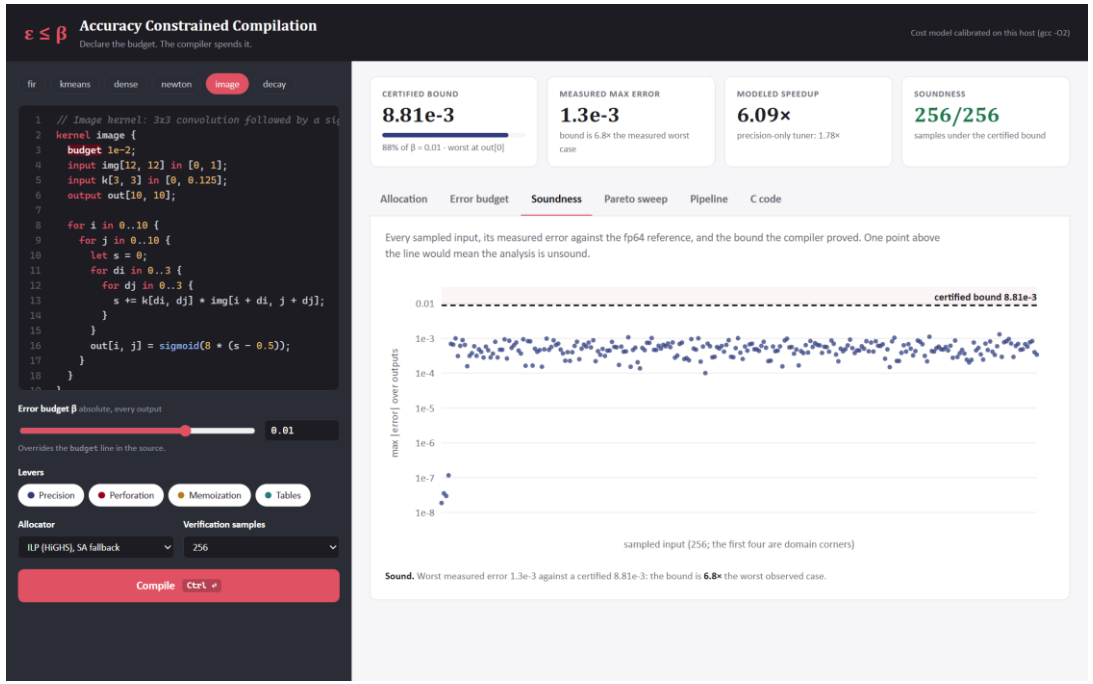
C code

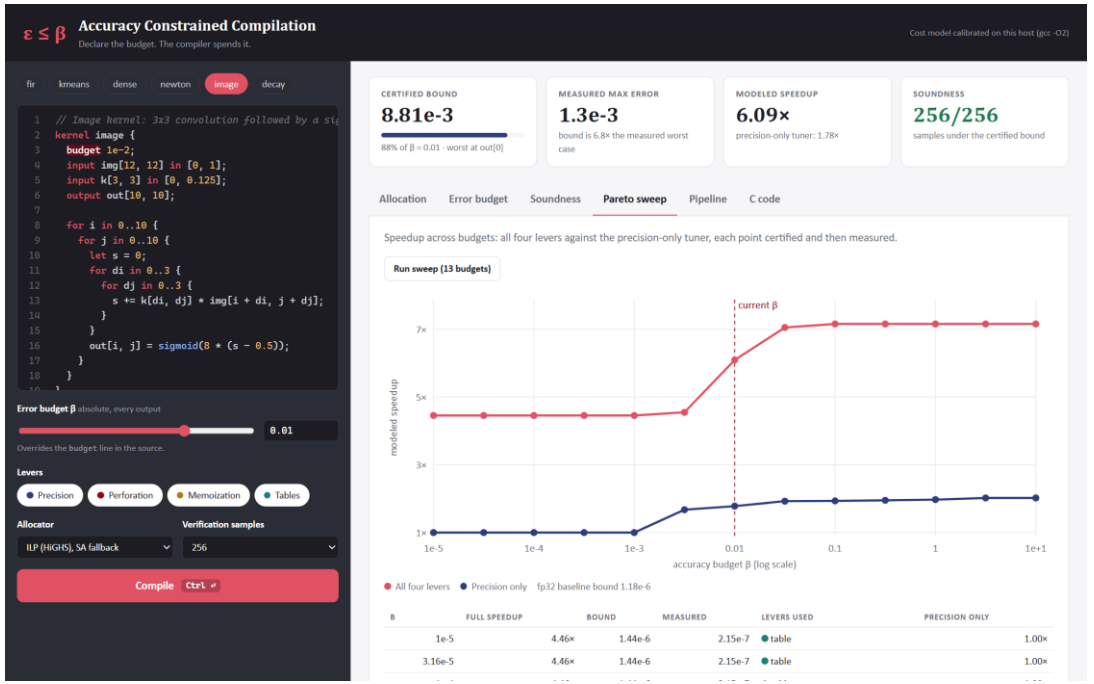
emitted kernel; one click compiles and times it with gcc

pip install flask numpy scipy · python acc/server.py · open http://127.0.0.1:5000

Demo: The Evidence Views

Soundness for the current compile, and the Pareto sweep, both generated on demand.





SOUNDNESS TAB

Image kernel, 256 samples; the first four are domain corners. Bound 8.8e-3, worst measured 1.4e-3.

PARETO TAB

Each point is allocated, certified and measured live, so the curve is regenerated for whatever kernel is in the editor.

Where We Arrived

REVIEW 1 ✓

Design locked

Language, IR, error model and menu specified.

REVIEW 2 ✓

Engine working

Front end, SSA, affine propagation validated by Monte Carlo.

REVIEW 3 ✓

Allocator working

Four levers, ILP allocation, first speedups.

FINAL ✓

Result defended

Soundness plot, Pareto curves, both baselines, native C.

Accuracy is a resource.
We built the compiler that spends it deliberately, proves every spend, and shows its work.

Thank you. Questions welcome.